

ZKFinger SDK

for Android

Version: 1.0.7

Date: Jul, 2020

ZKFinger SDK for Android

修订记录

日期	版本	描述	作者
2015-11-15	1.0.0	基础版本	陈建兴
2015-11-20	1.0.1	增加异步接口	陈建兴
2015-12-10	1.0.3	补充部分接口类	陈建兴
2019-06-10	1.0.4	合并 SDK	陈建兴
2019-10-22	1.0.5	3.4.12~3.4.17	陈建兴
2020-03-20	1.0.6	补充部分接口描述，算法错误码	陈建兴
2020-7-29	1.0.7	公司名变更	陈建兴

ZKFinger SDK for Android

感谢您使用熵基的ZKFinger SDK，在使用前请仔细阅读ZKFinger SDK概述，以便您能更快地掌握并使用ZKFinger SDK。

文档隐私权说明

非经过本公司书面同意，任何单位和个人不得擅自摘抄、复制本手册内容的部分或全部，并不得以任何形式传播。本手册中描述的产品中，可能包含我司及其可能存在的许可人享有版权的软件，除非获得相关权利人的许可，否则，任何人不得以任何形式对前述软件进行复制、分发、修改、摘录、反编译、反汇编、解密、反向工程、出租、转让等侵犯软件版权的行为。

文档使用说明

由于ZKFinger SDK软件功能不断扩充，ZKFinger SDK文档版本也会不断地升级，所以在使用ZKFinger SDK软件时，请详细阅读ZKFinger SDK文档内容。如有上诉原因给您造成的不便，敬请谅解，您也可以联系我们文档编写人，联系信息如下，谢谢！

1 ZKFinger SDK 概述

ZKFinger SDK是熵基提供给开发者的一套应用程序接口，具有统一管理各个设备模块的功能。开发者可以使用各个类中函数，开发操作熵基Android设备的应用。

ZKFinger SDK主要包括以下功能：

指纹图像采集：主要是操作指纹采集设备，如初始化设备、打开指纹采集设备，采集指纹图像，采集指纹模板，关闭设备等；

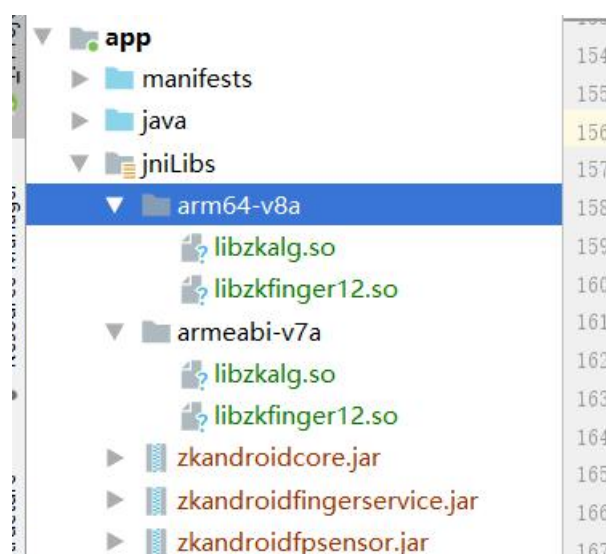
指纹算法识别：主要用于指纹比对识别，如初始化算法，释放算法，1:1，1:N等。

2 开发环境搭建

2.1 导入 zkandroidfpsensor.jar/zkandroidcore.jar/zkandroidfingerservice.jar

打开 ZKFinger SDK 文件夹，将 Device 目录中的 zkandroidfpsensor.jar/zkandroidcore.jar/zkandroidfingerservice.jar 导入到应用程序开发工具中（以 Android Studio 为例）。

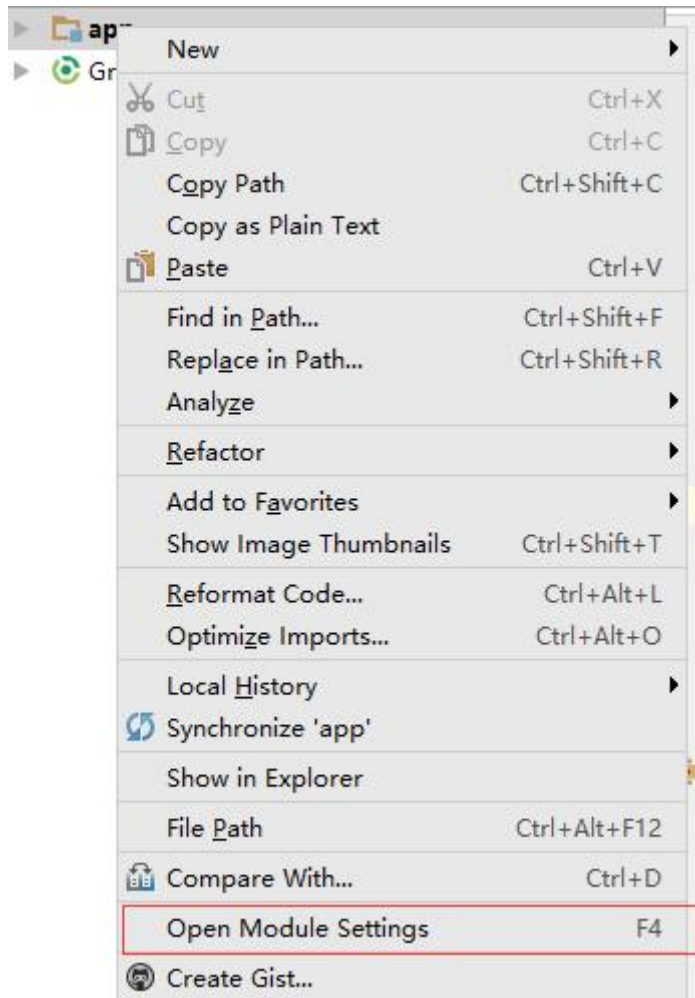
步骤 1：将 zkandroidfpsensor.jar、zkandroidfingerservice.jar 和 zkandroidcore.jar 拷贝到 app/libs 目录；



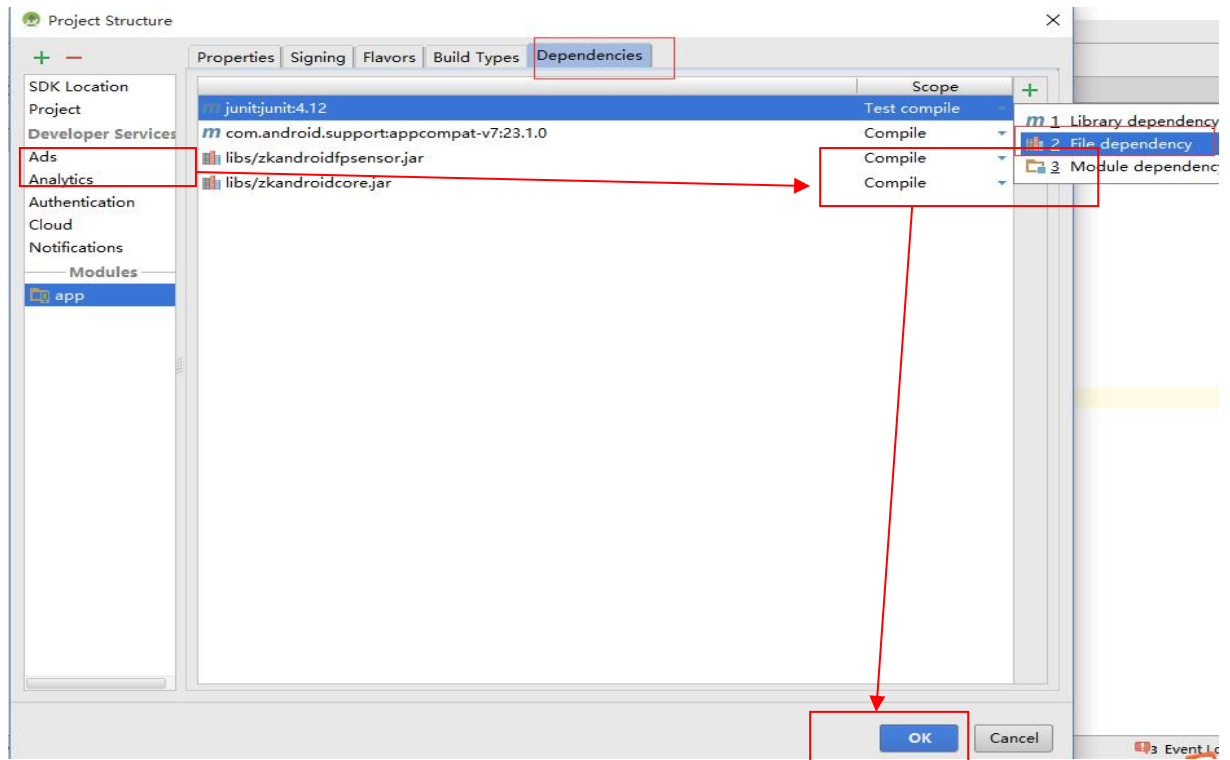
步骤 2：选择工程->Open Modules Setting->Dependencies->+ ->File dependency，选择 libs 文件夹里面的 zkandroidfpsensor.jar/ zkandroidcore.jar/zkandroidfingerservice.jar

ZKFinger SDK for Android

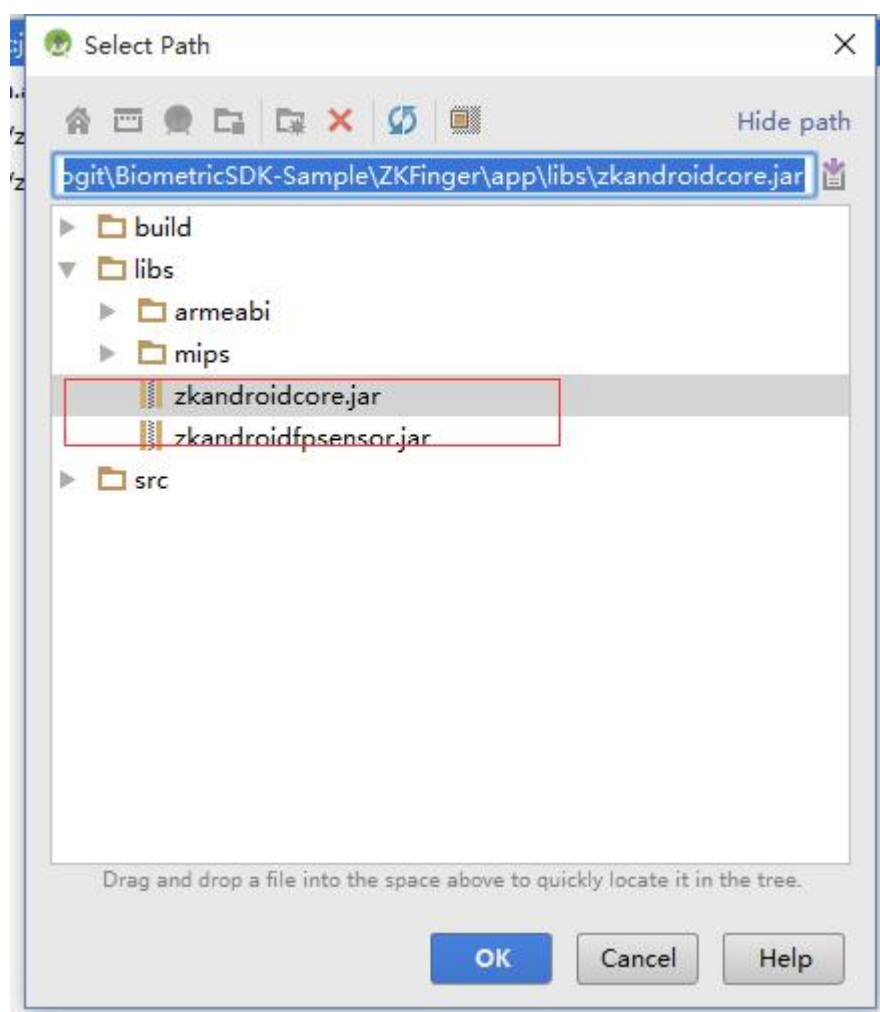
vice.jar->点击 OK->导入 zkandroidfpsensor.jar/ zkandroidcore.jar/zkandroidfingerser
vice.jar 成功。



ZKFinger SDK for Android



ZKFinger SDK for Android



3 ZKFinger SDK 设备接口

ZKFinger SDK 将各个功能模块抽象成类，用户通过调用类中方法完成对底层硬件设备的操作。

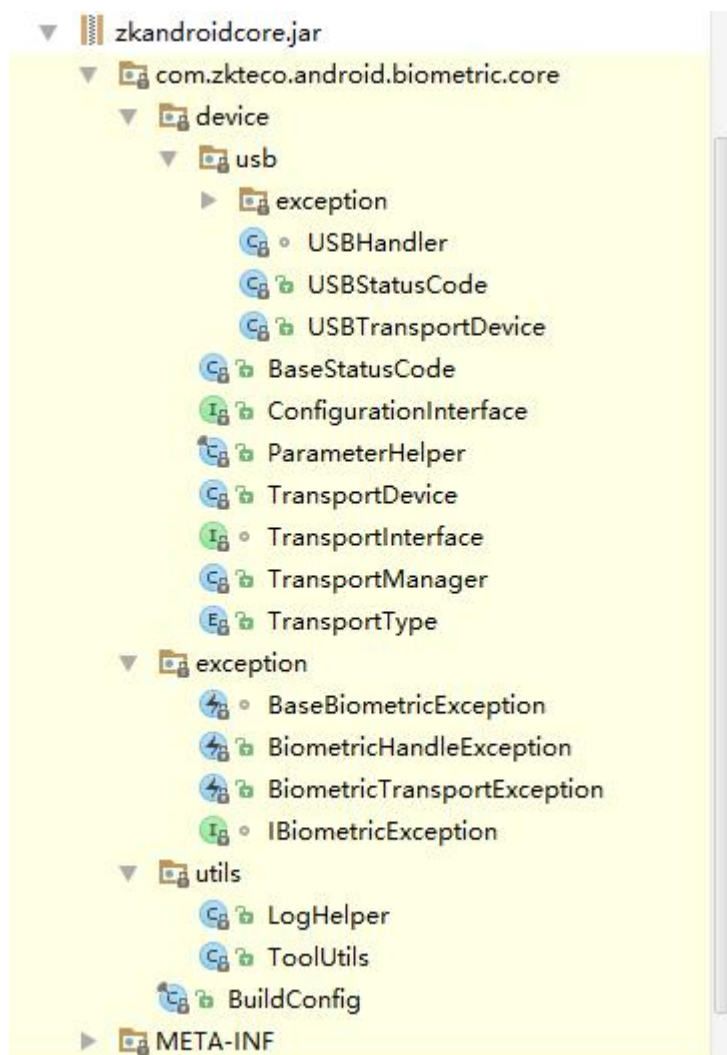
ZKFinger SDK 包括指纹采集设备类、控制设备类、常量类、异常处理类、算法处理类等。各类对应的类型如下图所示：

类名	类型
com.zkteco.android.biometric.module.fingerprint.FingerprintSensor	指纹采集设备类
com.zkteco.android.biometric.module.fingerprint.FingerprintStatusCode	错误码
com.zkteco.android.biometric.module.fingerprint.FingerprintCaptureListener	回调接口类
com.zkteco.android.biometric.module.fingerprint.FingerprintException	异常处理类

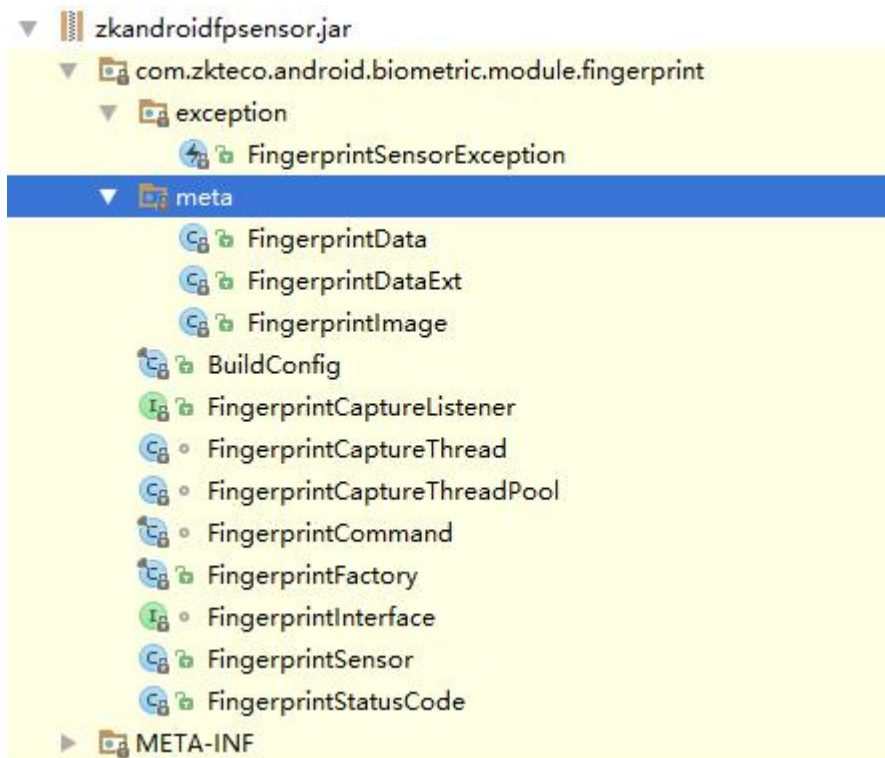
ZKFinger SDK for Android

exception.FingerprintSensorException	
com.zkteco.android.biometric.core.utils.LogHelper	日志工具类
com.zkteco.android.biometric.core.utils.ToolUtils	辅助工具类
com.zkteco.android.biometric.module.fingerprintreader.ZKFingerService	算法处理类

SDK 包结构如下:



ZKFinger SDK for Android



3.1 FingerprintSensor.class

FingerprintSensor.class 主要是操作指纹采集设备。如打开指纹采集设备，采集指纹图像，关闭设备等。调用流程请参考[指纹采集设备类流程图](#)。

3.1.1 open

【函数】

```
public void open(int index)
```

【功能】

打开指纹采集设备

【参数说明】

index

设备索引号，该值是接入采集器总数决定的。

例如：

当采集器总数为 1 时，则index的值为 0；

当采集器总数为 2 时，index的值为 0 或 1；

.....

【返回值】

无。

【注意】

错误码通过异常抛出。

通常场景只使用单一设备，index 传 0 即可

3.1.2 close

【函数】

```
public void close(int index)
```

【功能】

关闭设备

【参数说明】

index

参考open

【返回值】

无

【注意】

错误码通过异常抛出。

3.1.3 retrieveLastImage

【函数】

```
public int retrieveLastImage(int index, byte[] buffer, int[] imageAttributes)
```

【功能】

采集上一幅指纹图像。(startCapture后后台线程自动调用该函数，手指按压时主动调用FingerprintCaptureListener.captureOK返回图像和特征，不需要主动调用该函数)

【参数说明】

index

参考open

buffer

存储返回图像数据的数组。

buffer数组的大小建议预分配 480*640;

图像的宽度和高度在取像成功后获取。

imageAttributes

图像宽高数组

imageAttributes[0]存放图像宽

imageAttributes[1]存放图像高

【返回值】

大于0 图像数据长度

【注意】

acquireTemplate成功后获取上一幅指纹图像

错误码通过异常抛出

3.1.4 acquireTemplate

【函数】

public int acquireTemplate(int index, byte[] buffer)

【功能】

获取指纹模板。(startCapture后后台线程自动调用该函数，手指按压时主动调用FingerprintCaptureListener.captureOK 返回图像和特征，不需要主动调用该函数)

【参数说明】

index

参考Open

buffer

存储返回指纹模板数据的数组。

【返回值】

大于0 指纹模板数据长度

【注意】

错误码通过异常抛出

3.1.5 getLastTempLen

【函数】

public int getLastTempLen()

【功能】

获取最近一次指纹模板数据长度

【参数说明】

【返回值】

【注意】

可以在 FingerprintCaptureListener.captureOK 获取当前指纹模板数据长度

3.1.6 setFingerprintCaptureListener

【函数】

**public void setFingerprintCaptureListener(int index,
FingerprintCaptureListener listener)**

【功能】

设置回调函数。

【参数说明】

index

参考Open

FingerprintCaptureListener

回调接口

【返回值】

【注意】

异步取像，详见 Demo

3.1.7 setFingerprintCaptureMode

【函数】

public void setFingerprintCaptureMode(int index, int mode)

【功能】

设置采集模式（异步）

【参数说明】

index

参考 Open

mode

0 表示只取指纹模板(串口连接时建议只取模板，串口取像耗时超过 10s)

1 表示取指纹模板和指纹图像

【返回值】

【注意】

startCapture 成功后调用

3.1.8 startCapture

【函数】

public void startCapture(int index)

【功能】

开始取像(异步)。

【参数说明】

index

参考Open

[返回值]

[注意]

异步取像，通过 *setFingerprintCaptureListener* 设置的回调接口返回图像，模板。(详见 Demo)

3.1.9 stopCapture

[函数]

public void stopCapture (int index)

[功能]

停止取像(异步)。

[参数说明]

index

参考Open

[返回值]

[注意]

停止异步取像。

3.1.10 destroy

[函数]

public void destroy()

[功能]

销毁资源。

[参数说明]

无

[返回值]

[注意]

3.1.11 getFirmwareVersion

【函数】

public String getFirmwareVersion()

【功能】

获取固件版本

【参数说明】

无

【返回值】

固件版本号

【注意】

open成功后可获取

3.1.12 getSerialNumber

【函数】

public String getSerialNumber()

【功能】

获取设备序列号

【参数说明】

无

【返回值】

设备序列号

【注意】

open成功后可获取

3.1.13 rebootDevice

【函数】

public void rebootDevice(int index)

【功能】

重启指纹模块

【参数说明】

index

见**open**

【返回值】

无

【注意】

open成功后可调用

3.1.14 rebootDeviceEx

【函数】

public void rebootDeviceEx(int index)

【功能】

重启指纹模块

【参数说明】

index

见**open**

【返回值】

无

【注意】

无需先**Open**设备

3.1.15 version

【函数】

public String version()

【功能】

获取**SDK**版本

【参数说明】

无

【返回值】

SDK版本

【注意】

3.2 FingerprintCaptureListener.class

FingerprintCaptureListener.class 是异步取像接口类，调用方法请参考 Demo。

3.2.1 captureOK

[函数]

void captureOK(int captureMode, byte[] imageBuffer, int[]imageAttributes, byte[] templateBuffer);

[功能]

采集成功

[参数]

mode

0 表示只取指纹模板

1 表示取指纹模板和指纹图像

imageBuffer

存储返回图像数据的数组。（mode为 0 时指位null）

图像的宽度和高度在取像成功后获取。

imageAttributes

图像宽高数组（mode 为 0 时指位 null）

imageAttributes[0]存放图像宽

imageAttributes[0]存放图像高

templateBuffer

存储返回指纹模板数据的数组。

[返回值]

[注意]

3.2.2 captureError

[函数]

void captureError(FingerprintSensorException e)

[功能]

采集失败

[参数]

e,异常对象(见 3.3)

[返回值]

[注意]

3.3 FingerprintSensorException.class

FingerprintSensorException.class 为异常处理类。

3.3.1 getErrorCode

[函数]

```
public int getErrorCode()
```

[功能]

获取错误码

[参数]

[返回值]

错误码

[注意]

3.3.1 getErrorCode

[函数]

```
public int getInternalErrorCode ()
```

[功能]

获取内部（底层）错误码

[参数]

[返回值]

错误码

[注意]

3.4 ZKFingerService.class

指纹算法处理类

3.4.1 init

[函数]

```
public static native int init()
```

[功能]

初始化指纹识别引擎

[参数说明]

[返回值]

0 成功

否则 返回一个错误码，错误码说明请参考“[附录一](#)”

注：指纹识别引擎即指纹识别算法库。

3.4.2 free

[函数]

```
public static native int free()
```

[功能]

关闭指纹识别引擎，同时清空缓存中存储的所有指纹数据。

[参数说明]

无

[返回值]

0 成功

否则 返回一个错误码，错误码说明请参考“[附录一](#)”

3.4.3 verify

[函数]

```
static public int verify(byte[] temp1, byte[] temp2)
```

[功能]

比对两枚指纹模板

[参数]

temp1

第一个指纹模板数据数组

temp2

第二个指纹模板数据数组

[返回值]

指纹模板的匹配分数，返回值范围：0~1000。

建议：返回值大于所设置的阈值或默认值(50)时，可认为两个指纹模板匹配成功。

[注意]

错误码说明请参考“[附录一](#)”

3.4.4 verifyId

[函数]

static public int verifyId(byte[] temp, String id)

[功能]

将指纹模板与缓存中指定id的指纹模板进行匹配（即1:1比对），返回匹配分数。

[参数]

temp

指纹模板数据数组

id

被匹配的目标指纹模板id。id类型为字符串，字符串长度不能超过20个字节。

例如：String id = “ZKTeco_20_01”。

[返回值]

指纹模板的匹配分数，返回值范围：0~1000。

返回值大于所设置的阈值大小时或默认值(50)，表示两个指纹模板匹配成功。

3.4.5 identify

[函数]

static public int identify(byte[] temp, byte []idstr, int threshold, int count)

[功能]

将指纹模板数据与缓存中的指纹模板数据进行匹配（即1:N比对），返回匹配分数大于threshold的count个结果，若实际结果个数不足count，则返回实际结果个数。例如：count设置为2，返回值可能是0,1,2。

[参数]

temp

指纹模板数据数组

idstr

返回比对成功后的id号和分数的数组。

建议数组大小设置为50*count。

多个结果按照匹配分数从高到低的顺序排列,每个结果包括id和匹配分数，id和匹配分数之间通过' /t' 分隔，结果之间通过' /n' 分隔。

例如：String resultStrs= new String(idstr);

如resultStrs的返回值为：“1001' \t' 95' \n' 1002' \t' 85' \n' 1003' \t' 75”，则表示返回三个比对结果：

结果1： id: 1001, score: 95;

结果2： id: 1002, score: 85;

结果3： id: 1003, score: 75;

详细代码请查看[示例代码]。

threshold

匹配阈值，取值范围为0~1000（注:此值会覆盖先前所设置的阈值）。

匹配阈值是判断两个指纹特征模板相似程度的分界值，匹配阈值定得越高，表示对相似度的要求越严。两个指纹模板进行比对，返回的数值大于设置的阈值，表示比对成功，否则比对失败。通常在具体应用时可以根据情况灵活调整。

匹配阈值说明

拒判率	误判率	匹配阈值	
		1:N	1:1
高	低	80	60
中	中	75	55
低	高	70	50

count

期望返回的结果个数，建议设置为1。

例如：

threshold 设置为 70，**count** 设置为 3。如果指纹模板比对过程中只有 2 个模板符合匹配阈值，则函数返回值为 2。

[返回值]

成功 返回实际的比对结果个数。返回值可能小于**count**。1：表示1个指纹模板匹配成功。0：表示没有指纹模板匹配成功。

否则 返回一个错误码

[注意]

错误码说明请参考“[附录一](#)”

[示例代码]

```
int ret = 0;
byte[] idstr = new byte[50];
ret = ZKFingerServer.identify(templIdentify, idstr, 70, 1);
if( ret == 1 ){
    String resultString = new String(idstr);
    textView.setText("id: " + resultString.split("\t")[0] + ", score: " + resultString.split("\t")[1] );
}else{
    textView.setText("Please try again!");
}
```

3.4.6merge

[函数]

static public int merge(byte[] temp1, byte[] temp2, byte[] temp3, byte[] temp)

[功能]

合并 3 个指纹模板为登记模板

[参数]

temp1

待登记指纹模板 1

temp2

待登记指纹模板 2

temp3

待登记指纹模板 3

temp

输出登记指纹模板

[返回值]

>0 登记模板长度

其他 失败

[注意]

错误码说明请参考“[附录一](#)”

3.4.7 save

[函数]

static public save (byte[] temp, String id)

[功能]

将指纹模板数据保存到缓存。用户将自己数据库中需要用到的指纹数据，通过调用**save**方法保存到缓存中。

[注意]

调用**save**须确保指纹算法库已初始化（init）成功。

[参数]

temp

传入的指纹模板数据数组

id

指纹模板id。

id类型为字符串，字符串长度不能超过20个字节。

例如：String id = “ZKTeco_20_01”。

[返回值]

0 成功
否则 返回一个错误码

[注意]

错误码说明请参考“[附录一](#)”

3.4.8 get

[函数]

static public int get (byte[] temp, String id)

[功能]

获取缓存中指定 id 的指纹模板数据。

[参数]

temp

指定 id 的指纹模板数据的数组。建议将该数组大小设置为 1024*3，以确保有足够的存储空间储存指纹模板数据。

id

需要获取的指纹模板id。

id类型为字符串，字符串长度不能超过20个字节。

例如：String id = “ZKTeco_20_01”。

[返回值]

>0 成功，值为指纹模板长度
否则 返回一个错误码

[注意]

错误码说明请参考“[附录一](#)”

3.4.9 del

[函数]

static public int del(String id)

【功能】

从内存中删除登记的指纹模板

【参数】

id

需要删除的指纹模板的id。

id类型为字符串，字符串长度不能超过20个字节。

例如：String id = “ZKTeco_20_01”。

【返回值】

0 成功

否则 返回一个错误码

【注意】

错误码说明请参考“[附录一](#)”

3.4.10 clear

【函数】

static public int clear ()

【功能】

清空缓存中的所有指纹数据

【参数】

无

【返回值】

0 成功

否则 返回一个错误码

【注意】

错误码说明请参考“[附录一](#)”

3.4.11 count

【函数】

static public int count()

【功能】

获取缓存中当前存储的指纹模板数

【参数】

无

【返回值】

当前缓存中保存的指纹模板数

3.4.12 version

【函数】

static public void version(byte[] version)

【功能】

获取算法版本号

【参数】

version

返回算法版本号

【返回值】

3.4.13 setParameter

【函数】

static public int setParameter(int opCode, int value)

【功能】

设置算法参数

【参数】

opCode

参数代码(1 表示 1:1 阈值)

vaule

参数值(当 opCode=1 表示设置 1:1 阈值, 默认 50)

【返回值】

0 表示成功

其他表示失败

[备注]

除 **OpCode=1** 外请不要随意设置除非 **SDK** 开发人员额外说明

3.4.14 getTemplateQuality

[函数]

static public int getTemplateQuality(byte[] template)

[功能]

获取模板质量

[参数]

template

指纹模板

[返回值]

模板质量

3.4.15 getTemplateLength

[函数]

static public int getTemplateLength(byte[] template)

[功能]

获取模板长度

[参数]

template

指纹模板

[返回值]

模板数据有效长度

3.4.16 convertTemplate

[函数]

static public int convertTemplate(byte[] inTemp, byte[] outTemp)

[功能]

模板转换(12.0->10.0)

[参数]

inTemp

12.0 模板

outTemp

10.0 模板

[返回值]

> 0 表示转换成功，值为返回 10.0 模板数据长度

3.4.17 getFPLimitCount

[函数]

static public int getFPLimitCount()

[功能]

获取1:N 指纹容量

[参数]

无

[返回值]

1:N 指纹容量

3.5 实例代码(详见 Demo-Android Studio)

3.5.1 MainActivity.java-- 已替换换新的 demo 源码

```
package com.zkteco.slk20mdemo;

import androidx.annotation.RequiresApi;
import androidx.appcompat.app.AppCompatActivity;

import android.app.PendingIntent;
```

ZKFinger SDK for Android

```
import android.content.BroadcastReceiver;
import android.content.Context;
import android.content.Intent;
import android.content.IntentFilter;
import android.graphics.Bitmap;
import android.hardware.usb.UsbDevice;
import android.hardware.usb.UsbManager;
import android.os.Build;
import android.os.Bundle;
import android.util.Log;
import android.view.View;
import android.widget.ImageView;
import android.widget.TextView;

import com.zkteco.android.biometric.core.device.ParameterHelper;
import com.zkteco.android.biometric.core.device.TransportType;
import com.zkteco.android.biometric.core.utils.LogHelper;
import com.zkteco.android.biometric.core.utils.ToolUtils;
import com.zkteco.android.biometric.module.fingerprint.FingerprintCaptureListener;
import com.zkteco.android.biometric.module.fingerprint.FingerprintExceptionListene
r;
import com.zkteco.android.biometric.module.fingerprint.FingerprintFactory;
import com.zkteco.android.biometric.module.fingerprint.FingerprintSensor;
import com.zkteco.android.biometric.module.fingerprint.exception.FingerprintSensor
Exception;
import com.zkteco.android.biometric.module.fingerprintreader.ZKFingerService;

import java.util.HashMap;
import java.util.Map;

public class MainActivity extends AppCompatActivity {
    private static final int VID = 6997;    //zkteco device VID 6997
    private static final int PID = 289;
    private boolean mbStart = false;
```

ZKFinger SDK for Android

```
private TextView txtView = null;
private ImageView imageView = null;
private int index = 0;
private final static int ENROLL_COUNT    =    3;
private int enroll_index = 0;
private byte[][] regtemparray = new byte[3][2048]; //register template buffer
array
private boolean bRegister = false;
private static FingerprintSensor fingerprintSensor = null;

private void initUI()
{
    txtView = (TextView)findViewById(R.id.txtView);
    imageView = (ImageView)findViewById(R.id.imageView);
}

private void startFingerprintSensor() {
    if (null != fingerprintSensor)
    {
        return;
    }
    // Define output log level
    LogHelper.setLevel(Log.ASSERT);

    // Start fingerprint sensor
    Map fingerprintParams = new HashMap();
    //set vid
    fingerprintParams.put(ParameterHelper.PARAM_KEY_VID, VID);
    //set pid
    fingerprintParams.put(ParameterHelper.PARAM_KEY_PID, PID);
    fingerprintSensor = FingerprintFactory.createFingerprintSensor(getApplicationContext(), TransportType.USB, fingerprintParams);
}
```

ZKFinger SDK for Android

```
@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_main);
    initUI();
}

@Override
protected void onDestroy()
{
    super.onDestroy();
    if (null != fingerprintSensor) {
        FingerprintFactory.destroy(fingerprintSensor);
    }
}

private void setTextResult(String strResult)
{
    final String flStrResult = strResult;
    runOnUiThread(new Runnable() {
        public void run() {
            txtView.setText(flStrResult);
        }
    });
}

void doRegister(byte[] template, int templateLen)
{
    byte[] bufids = new byte[256];
    int ret = ZKFingerService.identify(template, bufids, 70, 1);
    if (ret > 0)
    {
```

ZKFinger SDK for Android

```
String strRes[] = new String(bufids).split("\t");
setTextResult("the finger already enroll by " + strRes[0] + ",cancel e
nroll");

bRegister = false;
enroll_index = 0;
return;
}

if (enroll_index > 0 && (ret = ZKFingerService.verify(regtemparray[enroll_
index-1], template)) <= 0)
{
    setTextResult("please press the same finger 3 times for the enrollment,
cancel enroll, socre=" + ret);
    bRegister = false;
    enroll_index = 0;
    return;
}

System.arraycopy(template, 0, regtemparray[enroll_index], 0, 2048);
enroll_index++;
if (enroll_index == ENROLL_COUNT) {
    byte[] regTemp = new byte[2048];
    int regTemplateLen = 0;
    if (0 < (regTemplateLen = ZKFingerService.merge(regtemparray[0], regte
mparray[1], regtemparray[2], regTemp))) {
        int retVal = 0;
        retVal = ZKFingerService.save(regTemp, "uid_" + (index++));
        if (0 == retVal)
        {
            setTextResult("enroll succ");
        }
        else
        {
            setTextResult("enroll fail, add template fail!");
        }
    } else {
```


ZKFinger SDK for Android

```
        setResult("enroll fail");
    }
    bRegister = false;
} else {
    setResult("You need to press the " + (3 - enroll_index) + "time fi
ngerprint");
}
}

void doIdentify(byte[] template, int templateLen)
{
    byte[] bufids = new byte[256];
    int ret = ZKFingerService.identify(template, bufids, 70, 1);
    if (ret > 0) {
        String strRes[] = new String(bufids).split("\t");
        setResult("identify succ, userid:" + strRes[0].trim() + ", score:"
+ strRes[1].trim());
    } else {
        setResult("identify fail, ret=" + ret);
    }
}

private void openDevice()
{
    startFingerprintSensor();
    try {
        bRegister = false;
        enroll_index = 0;
        //init zkfinger10
        int retVal = ZKFingerService.init();
        if (0 != retVal)
        {
            setResult("init zkfinger10 failed, ret=" + retVal);
            return;
        }
    }
}
```

ZKFinger SDK for Android

```
    }

    fingerprintSensor.open(0);

    FingerprintCaptureListener fingerprintCaptureListener = new Fingerprint
tCaptureListener() {

        @Override

        public void captureOK(int mode, byte[] rawImage, int[] attribs, by
te[] template) {

            if (mode == 1) //capture template and image
            {

                if (null != rawImage)
                {

                    final int width = attribs[0];
                    final int height = attribs[1];

                    final Bitmap bitmap = ToolUtils.renderCroppedGreyScale
Bitmap(rawImage, width, height);

                    runOnUiThread(new Runnable() {

                        public void run() {

                            imageView.setImageBitmap(bitmap);

                        }

                    });

                }

            }

            if (bRegister)
            {

                doRegister(template, fingerprintSensor.getLastTempLen());

            }

            else
            {

                doIdentify(template, fingerprintSensor.getLastTempLen());

            }

        }

        @Override

        public void captureError(FingerprintSensorException e) {
```

ZKFinger SDK for Android

```
        }

        };

        fingerprintSensor.setFingerprintCaptureListener(0, fingerprintCaptureL
istener);

        fingerprintSensor.startCapture(0);

        setResult("begin success, firmwareversion:" + fingerprintSensor.ge
tFirmwareVersion() + ", serial=" + fingerprintSensor.getSerialNumber());
    } catch (FingerprintSensorException e) {
        e.printStackTrace();
        try {
            fingerprintSensor.rebootDeviceEx(0);
        } catch (FingerprintSensorException ex) {
            //ex.printStackTrace();
        }
        setResult("open device failed!");
    }
}

public void onBnStart(View view)
{
    openDevice();
}

public void onBnEnroll(View view)
{
    bRegister = true;
    enroll_index = 0;
    setResult("Please press your finger 3 times(same finger.)");
}

private void closeDevice()
{
    if (mbStart)
```

```
{
    try {
        fingerprintSensor.stopCapture(0);
        fingerprintSensor.close(0);
    } catch (FingerprintSensorException e) {
        //e.printStackTrace();
    }
}

mbStart = false;
}

public void onBnStop(View view)
{
    closeDevice();
    setResult("device closed!");
}
}
```

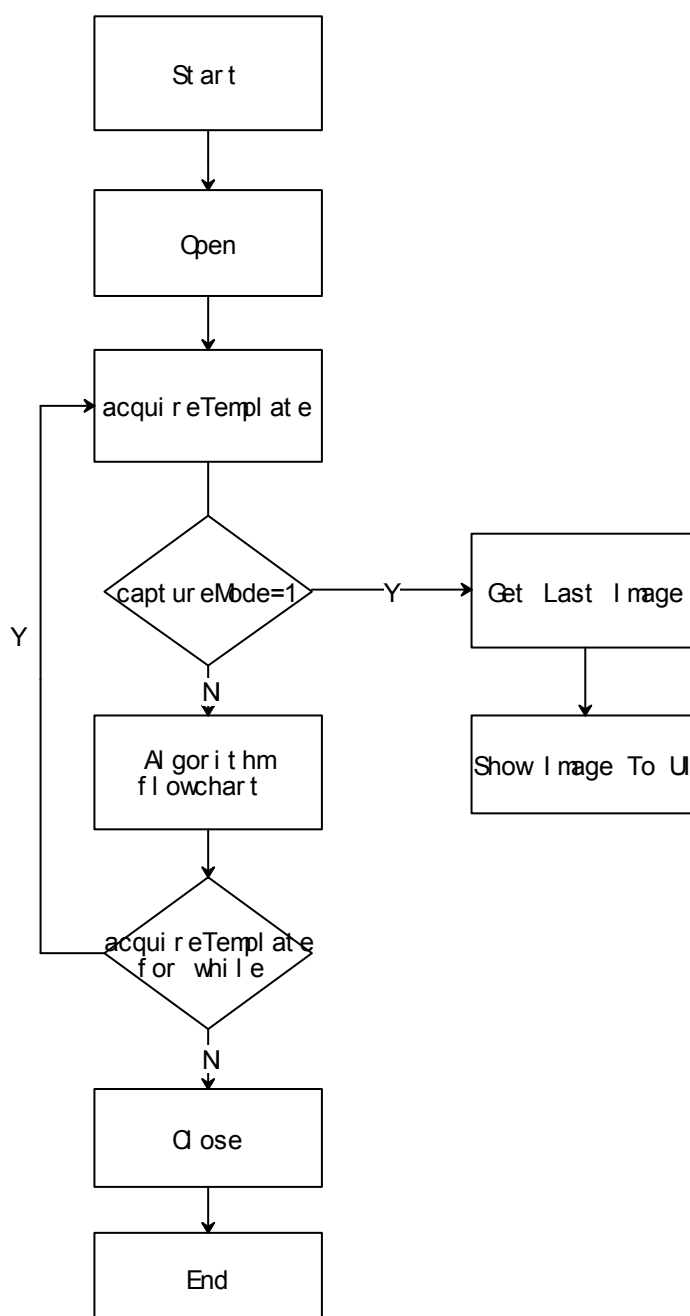
4 ZKFinger SDK 流程图

4.1 指纹采集设备类

指纹采集设备类：操作指纹采集器设备。如初始化指纹采集器、打开指纹采集器、获取指纹图像、获取指纹模板等。

4.1.1 指纹采集设备类流程图

以下流程以同步接口为例，异步接口调用详见 Demo。



步骤 1: 调用open函数，open的index是设备索引号，该值由当前接入设备总数决定的。

步骤 2: 开启一条线程，循环调用acuquireTemplate函数获取指纹模板，取模板成功后，进入“指纹算法流程图”（详见ZKFinger SDK for Android文档）。/或者开启异步取像模式。详细见Demo.

步骤 3: 调用 close 函数，关闭指纹采集器设备。

附录一

算法接口错误代码

错误码	备注
-5000	没有找到制定 id 指纹模板
-5002	参数错误
-5003	指纹模板错误
-5004	方法错误
-1116	算法未初始化
-1124	内部
-1128	容量已满
-1135	模板数据错误